PayWise AI – One-Day Build Workflow

Companion to: PRD v1.0 and TRD v1.0 | **Team:** 4 | **Build window:** one day (about 14 working hours plus buffer; times below are hours from kickoff, **T+0**)

---

1. Operating Rules

1. **Scope lock:** only P0 items from the PRD until the integration checkpoint at T+8 is green. P1/P2 only after that.
2. **Contract first:** API shapes in the TRD are final. Changes need a message in the team chat and an update to `docs/api-contracts.md` in the same PR.
3. **Mock until real:** the frontend builds against mock JSON that matches the contract; it switches to live endpoints as each one is merged.
4. **Deploy early:** hello-world frontend and backend must be live by T+3.
5. **Small PRs:** one feature per branch, merged within 2 hours, reviewed by a second person. `main` is always deployable.
6. **Feature freeze at T+11.** After that, bug fixes and demo prep only.
7. **Blocked for more than 20 minutes? Say so in chat.** Do not wait silently.

2. Roles and Ownership

Member	Role	Owns	Reviews
M1	Frontend + Integration lead	Design system, landing, auth screens, Send page, Dashboard, ExplainPanel, wiring to API	M2's frontend-facing endpoints
M2	Backend + Database	Supabase project, schema, RLS, auth validation, `/me`, `/recipients`, `/payments` (list/detail), `/notifications`, History page, Realtime	M3's schemas
M3	AI + Protection	Rule engine, issue detector, pre-flight endpoint, Gemini explainer, templates, unit tests	M4's status mapping

Member	Role	Owns	Reviews
M4	PayPal + Testing	Sandbox setup, Payouts call, webhook handler, state machine, simulator, Postman collection, deployment config, smoke tests	M1's UI against scenarios

Load balance note: the transaction History page and Pending timeline moved from M1 to M2 and M4 respectively, so M1 can focus on the Send flow and integration.

3. Timeline (hours from kickoff)

Phase 0: Align and Unblock (T+0 to T+2)

Who	Task	Output
All	Read PRD, TRD; confirm P0 scope in 15 minutes	Agreed scope
M1	Create GitHub repo, branch protection, folder skeleton, `.env.example`, README	Repo live, invites sent
M1	Vite + React app, routing, theme tokens, mock data file	Skeleton running
M2	Supabase project; run schema SQL; enable RLS and Realtime; share keys securely	Schema live, keys shared
M3	Finalize six-problem table; define pre-flight JSON; write test cases list	`rules` spec + test cases
M4	Create PayPal sandbox accounts: business, verified personal, unverified personal, and an email with no account; create the REST app; **check Payouts is enabled**	Credentials, go/no-go on Payouts

Gate G0 (T+2): repo, Supabase, PayPal credentials and the Payouts decision are all in place. If Payouts is not available, M4 announces the Orders v2 plus Simulator fallback now.

Phase 1: Foundations in Parallel (T+2 to T+6)

Who	Task	Output
M1	Landing, navigation, Login, Register (Supabase Auth); `api.js`, `useAuth`	Auth flow working
M1	Send page with PaymentForm and debounced RecipientCheck using mock pre-flight	Form works on mocks
M2	FastAPI app, CORS, `/health`, JWT validation, `/me`, `/recipients`; deploy to Render	Backend live
M2	Seed script: demo users, profiles, balances, saved recipients	Seed data
M3	`rules.py` for checks 1-4 with pytest; `POST /payments/preflight`	Pre-flight returns real results
M3	`templates.py` with all six fallback explanations	Templates ready
M4	`paypal/client.py` (OAuth), `payouts.py`, `status_map.py` with tests	Can create a sandbox payout
M4	Webhook route with signature verification; register webhook URL against the Render service	Webhook receiving events
M1	Deploy frontend to Vercel with env vars (by T+3)	Frontend live

Gate G1 (T+6): a logged-in user can open the Send page, see real pre-flight results, and the backend can create a sandbox payout and receive a webhook.

Phase 2: Integration (T+6 to T+8)

Who	Task
M4 + M2	`POST /payments` stores the transaction; webhook updates the status and writes `payment_events` and `notifications`
M1	Switch Send page from mocks to live endpoints; show RiskSummary; submit payment

Who	Task
M2	Realtime enabled on `transactions` and `notifications`; `GET /payments`, `/payments/{id}`
M3	Checks 5 and 6 in the engine; `POST /payments/{id}/explain` with Gemini and fallback

Gate G2, integration checkpoint (T+8): one real sandbox payment goes from the UI to PayPal to the webhook to a live dashboard update. **If G2 is not green, stop all P1 work and fix it together.**

Phase 3: Complete P0 and Scenarios (T+8 to T+11)

Who	Task
M1	Dashboard (balance, status counts, recent payments, alerts bell); Payment Detail with ExplainPanel and copy-message button
M2	History table with status filter; notifications read/unread; RLS review
M3	Explanation prompt tuning; caching; edge cases in rules; more unit tests
M4	Scenario Simulator route and buttons for problems 3, 5, 6 using PayPal-shaped payloads through the real handler; Postman collection committed
All	Seed the six demo scenarios from `docs/demo-scenarios.md`

Gate G3 (T+11), feature freeze: all P0 requirements demonstrable on the deployed app.

Phase 4: Polish, Test and Rehearse (T+11 to T+14)

Who	Task
M1	UI polish, empty and error states, Sandbox banner, optional landing page copy; P1 only if G3 is green

Who	Task
M2	Verify RLS with two accounts; check no data leaks; backup of schema SQL in repo
M3	Verify Gemini fallback by disabling the key; finalize wording of explanations
M4	Full smoke test on deployed URLs; set up uptime pinger; warm Render before the demo
All	Two full demo rehearsals with timer; assign who speaks and who drives the screen

Buffer (T+14 onward): reserved for submission tasks (README, architecture image, demo video if required, project submission form) and for fixing anything found in rehearsal.

4. Daily Rhythm

- **Stand-ups (5 minutes):** T+0, T+4, T+8, T+11. Each person answers: done, doing next, blocked.
- **Chat channel:** one place for decisions; pin the API contract, credentials location (never paste secrets in chat), and the demo script.
- **Review SLA:** a PR gets a first review within 30 minutes during the build window.
- **Credentials:** shared through a password manager or private note; each person keeps a local `.env`; never commit it.

5. Git Workflow

- **Branches:** `main` (protected, deployable); feature branches named `m1/send-page`, `m2/schema`, `m3/preflight`, `m4/webhooks`.
- **Commits:** short imperative messages, such as `Add preflight endpoint`.
- **PR checklist:** works locally; no secrets; contract file updated if shapes changed; loading and error states handled; reviewer assigned.
- **Merging:** M1 integrates, with M2 as the backup reviewer. If there is a conflict, the owner of the file resolves it.

- **After freeze:** only PRs labeled `fix`, merged by M1 after a quick test on the deployed preview.

6. Demo Scenarios (`docs/demo-scenarios.md`)

#	Scenario	Input	Expected pre-flight	Expected status	Expected explanation
1	Wrong email	`sam@gmial.com` (saved contact is `sam@gmail.com`)	High risk: did you mean sam@gmail.com	blocked before sending	Suggests the correction
2	No PayPal account	Sandbox email with no account	Medium: may become unclaimed	`unclaimed` (real webhook)	Recipient must sign up with this exact email; funds return after the holding period
3	Incomplete verification	Recipient profile unverified	Warn	`on_hold`	Recipient must complete verification
4	Insufficient balance	Amount greater than balance	Fail, cannot proceed	not created	Shows the shortfall
5	Manual acceptance	Simulator	n/a	`pending` with reason	Recipient must accept payment manually
6	eCheck / bank delay	Simulator	n/a	`pending` with reason	Bank-funded payment clears in several business days
7	Happy path	Verified recipient, valid amount	Low	`completed` (real webhook)	Payment delivered

7. Priority Cut Lines

If the day runs late, cut in this order, keeping the core story intact:

1. Email notifications (P2), currency warning (P2)
2. Landing page polish, Pending timeline (P1)
3. History table filter (P1)

4. Replace real-webhook demo of problem 3 with the simulator

Never cut: pre-flight, one real sandbox payment with a real webhook, live dashboard update, Gemini explanation with fallback, the simulator.

8. Submission Checklist

- [ ] Frontend and backend deployed; demo URLs tested on a fresh browser
- [ ] README: problem, solution, architecture diagram, stack, setup steps, env var names, demo accounts, known limitations
- [ ] Architecture diagram updated (Notifications as in-app, Elastic removed, simulator shown)
- [ ] PRD, TRD and this workflow committed under `docs/`
- [ ] Postman collection committed
- [ ] Demo script rehearsed twice; backup screen recording saved
- [ ] Hackathon submission form filled with repo and demo links
- [ ] Honest limitations slide: sandbox only, simulated balance, email-account lookup is a prediction

9. Daily Risk Watchlist

Risk	Owner	Trigger	Action
Payouts not enabled	M4	G0	Switch to Orders v2 plus Simulator
Webhook not reaching Render	M4	T+6	Check URL and webhook ID; log raw requests; polling fallback
Gemini failures	M3	any	Templates already ship by T+6
Realtime not updating	M2	T+8	Polling every 5 seconds
Merge conflicts	M1	T+9 onward	Smaller PRs; freeze at T+11
Render cold start	M4	demo	Pinger plus warm-up